services\cache\cache_manager.py

```
import threading
import time
from datetime import datetime, timezone
from typing import Dict, List, Any, Optional
import json
import gc
import config
from database.db_manager import db_manager

class CacheManager:
"""Lightweight caching system with database persistence for reduced memory footprint"""
def __init__(self):
self._cache_timestamps = {}
self._cves_30_days: List[Dict[str, Any]] = []
self._historical_data: Dict[int, List[Dict[str, Any]]] = {}
self._historical_loaded = False
self._lock = threading.Lock()
self._last_memory_check = time.time()
self._memory_check_interval = 60  # seconds
self.API_CACHE_DURATION = 24 * 60 * 60  # 24 hours
self.HISTORICAL_CACHE_DURATION = 7 * 24 * 60 * 60  # 7 days
self.db_manager = db_manager
print("[Cache] CacheManager initialized with memory threshold of",
config.MAX_MEMORY_USAGE_MB, "MB")

# ------------------ 30-day API CVE cache ------------------
def get_api_data_30_days(self) -> Optional[List[Dict[str, Any]]]:
"""Return cached 30-day CVEs if valid, else None"""
cache_key = "api_30_days"
# Check DB first
if self.db_manager.use_database:
cached = self.db_manager.get_cves_by_filter(limit=2000)
if cached:
print(f"[Cache] Using database cache: {len(cached)} CVEs")
return cached

# Check memory cache
with self._lock:
if self._cves_30_days and cache_key in self._cache_timestamps:
age = time.time() - self._cache_timestamps[cache_key]
if age < self.API_CACHE_DURATION:
print(f"[Cache] Using in-memory cache: {len(self._cves_30_days)} CVEs")
return self._cves_30_days
return None

def set_api_data_30_days(self, data: List[Dict[str, Any]]):
"""Set cache for 30-day CVEs"""
cache_key = "api_30_days"
with self._lock:
self._cves_30_days = data
self._cache_timestamps[cache_key] = time.time()

# Save metadata to DB
if self.db_manager.use_database:
self.db_manager.update_cache_metadata(
cache_key,
len(data),
{"last_check": datetime.now(timezone.utc).isoformat(), "type": "api_30_days"}
)
print(f"[Cache] Cached {len(data)} CVEs for last 30 days")

# ------------------ Historical data cache ------------------
def get_historical_data(self, year: int) -> Optional[List[Dict[str, Any]]]:
"""Return historical CVEs for a specific year"""
with self._lock:
if year in self._historical_data:
return self._historical_data[year]
return None

def set_historical_data(self, year: int, data: List[Dict[str, Any]]):
"""Cache historical CVEs for a specific year"""
with self._lock:
```

```
self._historical_data[year] = data
self._historical_loaded = True
if self.db_manager.use_database:
self.db_manager.save_historical_cves(year, data)
print(f"[Cache] Cached {len(data)} historical CVEs for {year}")

def is_historical_loaded(self) -> bool:
"""Check if any historical data is loaded"""
if self.db_manager.use_database and
self.db_manager.get_cache_metadata("historical_loaded"):
return True
return self._historical_loaded

# ------------------ Summary stats ------------------
def get_summary_stats(self, key: str) -> Optional[Dict[str, Any]]:
if self.db_manager.use_database:
return self.db_manager.get_summary_stats(key)
return None

def set_summary_stats(self, key: str, count: int, data: Dict[str, Any]):
if self.db_manager.use_database:
self.db_manager.save_summary_stats(key, count, data)
print(f"[Cache] Saved summary stats for {key} ({count} records)")

# ------------------ Timeline data ------------------
def get_timeline_data(self, years=1) -> Optional[Dict[str, Any]]:
if self.db_manager.use_database:
return self.db_manager.get_timeline_data(years)
return None

def set_timeline_data(self, data: Dict[str, Any]):
if self.db_manager.use_database and 'raw_data' in data:
year_month_counts = {}
for date_str, count in data['raw_data'].items():
parts = date_str.split('-')
if len(parts) >= 2:
year = int(parts[0])
month = int(parts[1])
year_month_counts[(year, month)] = count
self.db_manager.save_timeline_data(year_month_counts)
print(f"[Cache] Saved timeline data ({len(year_month_counts)} records)")

# ------------------ Cache stats ------------------
def get_cache_stats(self) -> Dict[str, Any]:
with self._lock:
stats = {
'cached_timestamps': len(self._cache_timestamps),
'historical_loaded': self._historical_loaded,
'cache_ages': {},
'database_enabled': self.db_manager.use_database
}
now = time.time()
for key, ts in self._cache_timestamps.items():
stats['cache_ages'][key] = f"{int((now - ts)/60)} minutes"
if self.db_manager.use_database:
stats.update(self.db_manager.get_stats())
return stats

# ------------------ Clear cache ------------------
def clear_cache(self):
with self._lock:
self._cves_30_days.clear()
self._historical_data.clear()
self._historical_loaded = False
self._cache_timestamps.clear()
if self.db_manager.use_database:
self.db_manager.clear_all_cves()
print("[Cache] All caches cleared")

def clear_all(self):
"""Clear all caches including DB"""
self.clear_cache()
print("[Cache] All caches cleared including database")

# ------------------ Memory optimization ------------------
def check_memory_usage(self):
now = time.time()
```

```
if now - self._last_memory_check < self._memory_check_interval:
return
self._last_memory_check = now
try:
import psutil, os
process = psutil.Process(os.getpid())
mem_mb = process.memory_info().rss / 1024 / 1024
print(f"[Cache] Current memory usage: {mem_mb:.2f} MB")
if mem_mb > config.MAX_MEMORY_USAGE_MB:
print(f"[Cache] Memory usage high ({mem_mb:.2f} MB), clearing in-memory caches")
with self._lock:
self._cves_30_days.clear()
self._historical_data.clear()
self._historical_loaded = False
gc.collect()
except Exception as e:
print(f"[Cache] Memory check failed: {e}")

# ------------------ Warm-up ------------------
def warm_up_cache(self):
if self.db_manager.use_database:
print("[Cache] Warming up cache using database (no-op)")
return True
print("[Cache] Cache warm-up skipped")
return False

# ------------------ Global instance ------------------
cache_manager = CacheManager()
```